minicut 動画編集エディタ

組み込み JavaScript & プラグイン API リファレンスマニュアル

Duktape 2.7.0 エンジン 完全な UTF-8 / 多言語対応 ドッキング可能ツールパネル ツールシェルフドック

ネットワークライブラリ (libcurl)

1. はじめに & JavaScript アーキテクチャ

minicut は完全な **JavaScript** エンジン（Duktape 2.7.0）を内蔵しており、標準でネイティブ UTF-8 および多言語 Unicode 文字列を扱えます。 Lua と完全に同等の API が提供されており、日本語などのスクリプトツールを容易に構築できます。

スクリプト配置場所

配置場所	役割	説明
`~/.config/minicut/autorun/`	起動時自動実行	`.js` ファイルは起動時に自動読み込みされ、「ドック」にアイコンが追加されます。
`~/.config/minicut/scripts/`	メニュー実行	`.js` ファイルは「モディファイア (Modifiers)」メニューに表示されます。
`~/.config/minicut/shaders/gl/`	カスタム GL シェーダー	OpenGL 頂点/フラグメントシェーダー。

配置場所	役割	説明
`~/.config/minicut/shaders/vulkan/`	カスタム Vulkan シェーダー	Vulkan 用 GLSL シェーダー。
`~/.config/minicut/icons/`	ドックアイコン	スクリプト名と一致する PNG アイコン画像です。

2. JavaScript プラグイン構成

```
var targetTrack = 0;

var clipDuration = 4.0;

var mediaPath = "tests/assets/test_sample.png";



function placeMedia() {

    minicut.push_undo();

    var playhead = timeline.get_playhead();

    var cid = timeline.place_clip(mediaPath, targetTrack, playhead, clipDuration);

    if (cid && cid.length > 0) {

        clip.set_scale(cid, 1.25);

        timeline.add_keyframe(cid, "opacity", 0.0, 0.0);

        timeline.add_keyframe(cid, "opacity", 1.0, 1.0);

        minicut.set_status("クリップを配置しました: " + cid);

    }

}



function browseFile() {
```

```
    gui.file_dialog("ファイルを選択", function(selected) {
        if (selected && selected.length > 0) {
            mediaPath = selected;
            minicut.set_status("選択: " + selected);
        }
    });
}

plugin.init_panel = function() {
    gui.text_colored(0.24, 0.68, 0.91, 1.0, "メディア配置ツール (JS)");
    gui.separator();
    targetTrack = gui.slider_int("トラック番号", targetTrack, 0, 4);
    clipDuration = gui.slider_float("長さ(秒)", clipDuration, 0.5, 30.0);
    gui.button("ファイル参照...", browseFile);
    gui.button("タイムラインに配置", placeMedia);
};
```

3. コアモジュール API

JavaScript から利用可能な主要モジュールは `project`, `timeline`, `clip`, `gui`, `net`, `minicut`, `export` です。

3.1. クリップ変形 & 属性 (`clip`)

関数	戻り値	説明
`clip.get_position(id)`	`array [x, y]`	中心基準の (x, y) 座標を取得します。
`clip.set_position(id, x, y)`	`void`	中心座標を設定します。

関数	戻り値	説明
`clip.get_scale(id)`	`number`	スケール倍率（1.0 = 100%）を取得します。
`clip.set_scale(id, s)`	`void`	スケール倍率を設定します。
`clip.get_opacity(id)`	`number`	不透明度 [0.0, 1.0] を取得します。
`clip.set_opacity(id, o)`	`void`	不透明度を設定します。
`clip.get_volume(id)`	`number`	音声ボリュームゲイン [0.0, 2.0] を取得します。
`clip.set_volume(id, v)`	`void`	音声ボリュームゲインを設定します。
`clip.get_transform(id)`	`object`	変形オブジェクト `{x, y, scale, rotation}` を取得します。
`clip.set_transform(id, x, y, s, r)`	`void`	位置(X, Y)、拡大率、回転角を一括設定します。
`clip.reset_transform(id)`	`void`	変形を初期状態（x=0, y=0, scale=1.0, rotation=0）にリセットします。

3.2. GUI コントロール (`gui`)

関数	戻り値	説明
`gui.text(text)`	`void`	テキストを表示します。
`gui.text_colored(r, g, b, a, text)`	`void`	色付きテキストを表示します。
`gui.button(label, [callback])`	`boolean`	ボタンを表示しクリック時に実行します。
`gui.slider_float(label, val, min, max)`	`number`	浮動小数点スライダーを表示します。

関数	戻り値	説明
`gui.slider_int(label, val, min, max)`	`integer`	整数スライダーを表示します。
`gui.input_text(label, text)`	`string`	テキスト入力フィールドを表示します。
`gui.file_dialog(title, cb, [filter])`	`void`	ファイルダイアログを開き結果を返します。
`gui.popout_preview(popped)`	`void`	プレビュー画面を独立ウィンドウに分離（true）またはドッキング（false）します。
`gui.is_preview_popped_out()`	`boolean`	プレビューが現在分離されているかを取得します。
`gui.set_preview_aspect_mode(mode)`	`void`	アスペクト比モード（0:自動, 1:16:9, 2:9:16, 3:4:3, 4:1:1, 5:21:9, 6:4:5）を設定します。
`gui.get_preview_aspect_mode()`	`number`	現在のアスペクト比モード（0〜6）を取得します。
`gui.set_transform_tool(tool)`	`void`	ギズモ変形ツール（"select", "translate", "scale", "rotate"）を設定します。
`gui.get_transform_tool()`	`string`	現在の変形ツール名を取得します。

3.3. エクスポート操作 & 進捗状況 (`export`)

関数	戻り値	説明
`export.get_audio_progress()`	`number`	音声ミキシングの進捗率（ステレオ・キーフレーム処理、0.0〜1.0）を取得します。
`export.get_progress()`	`number`	全体のエクスポート進捗率（0.0〜1.0）を取得します。
`export.is_exporting()`	`boolean`	エクスポート処理が実行中かを取得します。

関数	戻り値	説明
`export.get_status()`	`string`	パイプラインの現在の状態文字列を取得します。